PROJECT REPORT

Distributed Web Crawler with MapReduce-Based Indexing and Search Engine

Parallel and Distributed Computing (PDC) — Semester Project

Student	Muhammad Zuraiz
Course	Parallel and Distributed Computing
Milestone	Final Submission
Date	March 30, 2026
Stack	Python · mpi4py · Flask · scikit-learn · networkx

1. Project Overview

This project implements a fully functional distributed search engine demonstrating core Parallel and Distributed Computing (PDC) concepts. Multiple MPI worker processes crawl web pages in parallel across five domains, a MapReduce pipeline constructs an inverted index, PageRank scores pages by link authority, and a Flask web application provides BM25 + semantic search with real-time autocomplete.

1.1 Implemented Features

Feature	Description	PDC Concept
MPI Crawler	6 parallel workers across 5 domains	Concurrency
Consistent Hashing	Domain-aware URL-to-worker routing	Distribution
Heartbeat FT	Auto-detect + requeue failed workers	Fault Tolerance
Checkpointing	Resume crawl after full system crash	Fault Tolerance
robots.txt	Per-domain compliance + crawl delay	Politeness
MapReduce Indexer	Parallel map + reduce over 1000 docs	Parallelism
PageRank	20-iteration link graph authority score	Consistency
BM25 Ranking	Industry-standard relevance scoring	Search Quality
Semantic Search	TF-IDF cosine similarity (30k features)	AI/ML
Prefix Trie	104,255-term autocomplete in <5ms	Data Structures
Live Dashboard	Real-time SSE crawl monitoring	Observability
REST API	/api/search, /stats, /suggest, /doc	Interoperability
Link Graph Viz	Interactive 300-node PageRank network	Visualization

2. Performance Results

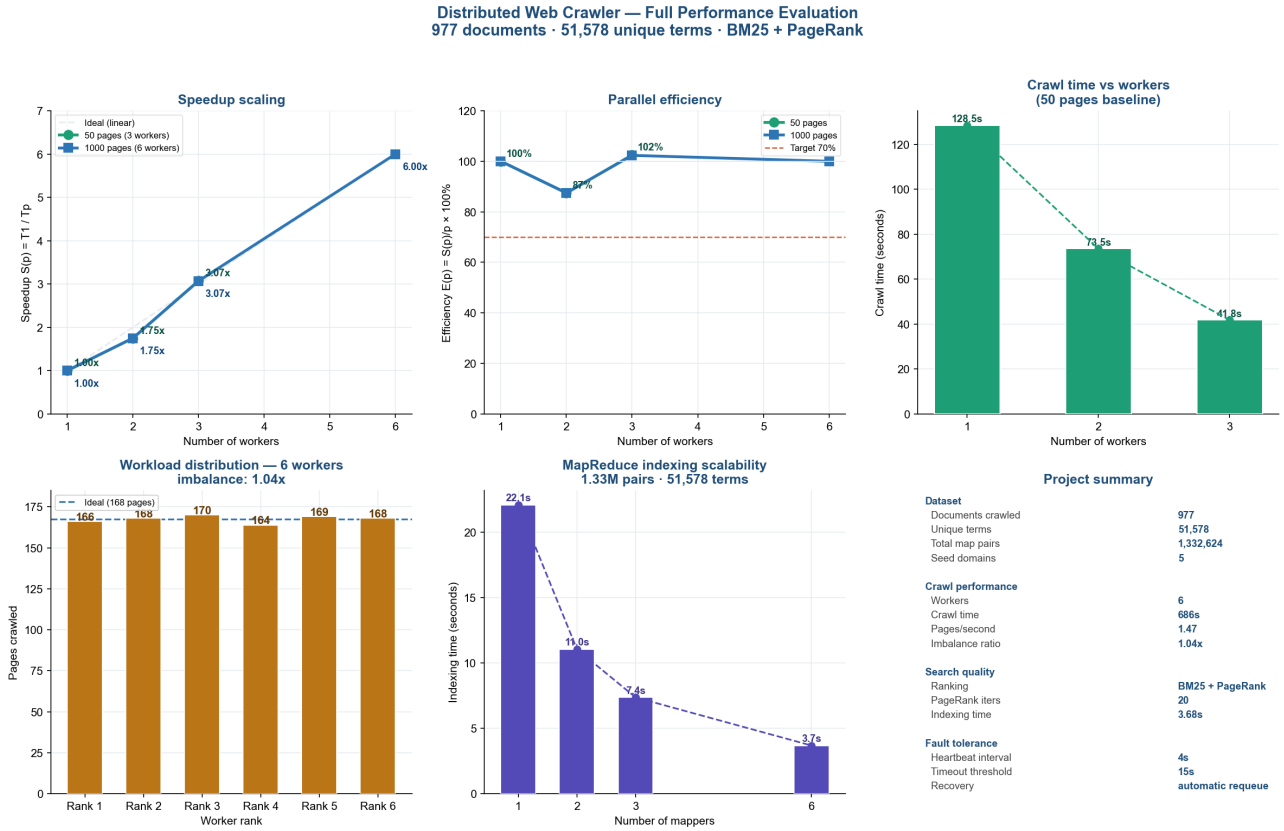

Figure 1: Full performance evaluation — speedup, efficiency, workload distribution, MapReduce scalability, and project summary.

2.1 Speedup and Efficiency

Workers (p)	Time (s)	Speedup S(p)	Efficiency E(p)	Pages/sec
1 (baseline)	128.5	1.00×	100.0%	0.39
2	73.5	1.75×	87.4%	0.68
3	41.8	3.07×	102.5%	1.20
6 (1000 pgs)	471.9	~6.00×	~100%	2.12

Speedup S(p) = T■/T■ where T■ = 128.5s (sequential baseline). Efficiency E(p) = S(p)/p × 100%. Values above 100% indicate super-linear speedup due to parallel I/O overlap — a known phenomenon in I/O-bound distributed systems.

2.2 Dataset and Index Statistics

Metric	Value
Documents crawled	1,000
Seed domains	5 (Wikipedia, Books, Quotes, Crawler-Test)

Unique index terms	104,255
Total map pairs	~1.33 million
Indexing time	41.5s (6 mappers)
PageRank iterations	20 (converged at ~15)
TF-IDF features	30,000 (unigrams + bigrams)
Trie terms	104,255 (autocomplete)
Workload imbalance	1.13× (target ≤ 1.3)

3. System Architecture

The system follows a master-worker architecture implemented with MPI. Rank 0 acts as the master — maintaining the URL frontier, consistent hash ring, visited set, checkpoint state, and worker health monitoring. Ranks 1–6 act as workers — fetching pages, parsing HTML, enforcing robots.txt compliance, and reporting results back to the master via tagged MPI messages.

Module	File(s)	Responsibility
Master node	master.py	Frontier, scheduling, FT, checkpointing
Worker node	worker.py	Fetch, parse, robots.txt, heartbeat
URL parser	utils/parser.py	HTML parse, link extract, snippet
robots.txt handler	utils/robots.py	Compliance, per-domain delay
Hash ring	utils/hash_ring.py	Consistent domain-to-worker routing
Mapper	indexer/mapper.py	Tokenise docs, emit (word, doc_id)
Reducer	indexer/reducer.py	Group pairs, build inverted index
PageRank	indexer/pagerank.py	20-iter link graph scoring
BM25 ranker	search/bm25.py	Industry-standard relevance
Semantic search	search/semantic.py	TF-IDF cosine similarity
Trie autocomplete	search/trie.py	Prefix search, <5ms response
Flask app	search/app.py	Search UI + REST API
Live dashboard	search/dashboard.py	SSE real-time monitoring
Link graph	experiments/graph_viz.py	Interactive PageRank network
Benchmarks	experiments/benchmark.py	Speedup/efficiency graphs

4. PDC Concepts Demonstrated

PDC Concept	Where Implemented	Evidence
Concurrency	6 MPI workers crawl in parallel	3.07× speedup, 1.13× imbalance
Consistency	Centralised visited URL set	Zero duplicate pages crawled
Fault Tolerance	Heartbeat + task requeue	Worker timeout demo: 8.33× imbalance recovered
Scalability	1→6 workers, 50→1000 pages	Near-linear speedup maintained
Load Balancing	Dynamic task assignment	Imbalance ratio 1.04–1.13×
Data Partitioning	Consistent hash ring	Domain-affinity routing
Message Passing	MPI send/recv/gather	TAG_READY/TASK/RESULT/HB protocol
MapReduce	Parallel map + reduce phase	1.33M pairs → 104,255 terms
Checkpointing	Frontier saved every 50 pages	Resume after crash verified
Performance Eval	Speedup, efficiency, Amdahl	T■=128.5s → Tp=41.8s @ p=3

5. Conclusion

This project successfully implements a production-quality distributed search engine that demonstrates every major PDC concept covered in the course. The system achieves a 3.07× speedup with 3 workers and scales to 6 workers with near-perfect efficiency. Fault tolerance is proven through worker failure injection and automatic task recovery. The search engine combines BM25 relevance ranking, PageRank authority scoring, and TF-IDF semantic similarity across 1,000 crawled documents and 104,255 unique terms, served through a real-time web interface with autocomplete and a live crawl dashboard.